PROJECT REPORT

CLASSROOM TIMETABLE GENERATOR

Submitted By:

Khushi Kumari

Course:

Bachelor of Technology (B.Tech)

Branch:

Computer Science and Engineering (CSE Core)

College:

Rungta International Skills University, Bhilai

Programming Language:

Python 3

Academic Year:

2026–2027

1. INTRODUCTION

A timetable is an important part of every educational institution. It helps students and teachers manage their time properly.

The Classroom Timetable Generator is a Python-based project that automatically generates a weekly class timetable. It distributes subjects across different days and periods according to the given rules.

This project uses Python concepts such as lists, dictionaries, loops, conditional statements, and scheduling logic.

The main purpose of this project is to create a valid timetable while following the given constraints.

2. PROBLEM STATEMENT

The problem is to develop a Python program that generates a weekly classroom timetable automatically.

The timetable must follow these conditions:

1. The total number of periods must be equal to the number of days multiplied by periods per day.
2. A subject can be scheduled at most two times in a single day.
3. The subject with the highest number of remaining periods should be selected first.
4. If two subjects have the same remaining periods, the subject entered first should be selected.

The program should generate a timetable according to these rules and display an error message if the given input is invalid.

3. OBJECTIVES

The main objectives of the Classroom Timetable Generator project are:

1. To generate a weekly classroom timetable automatically.
2. To save time and reduce manual work.
3. To distribute subjects among different days and periods.
4. To ensure that a subject is scheduled at most two times per day.
5. To follow the given scheduling rules.
6. To display a valid timetable or an error message for invalid input.
7. To understand Python programming concepts through a practical project.

4. SOFTWARE REQUIREMENTS

The following software and hardware are required to develop and run this project.

Hardware Requirements:

1. Computer or Laptop
2. Minimum 4 GB RAM
3. Keyboard and Mouse

Software Requirements:

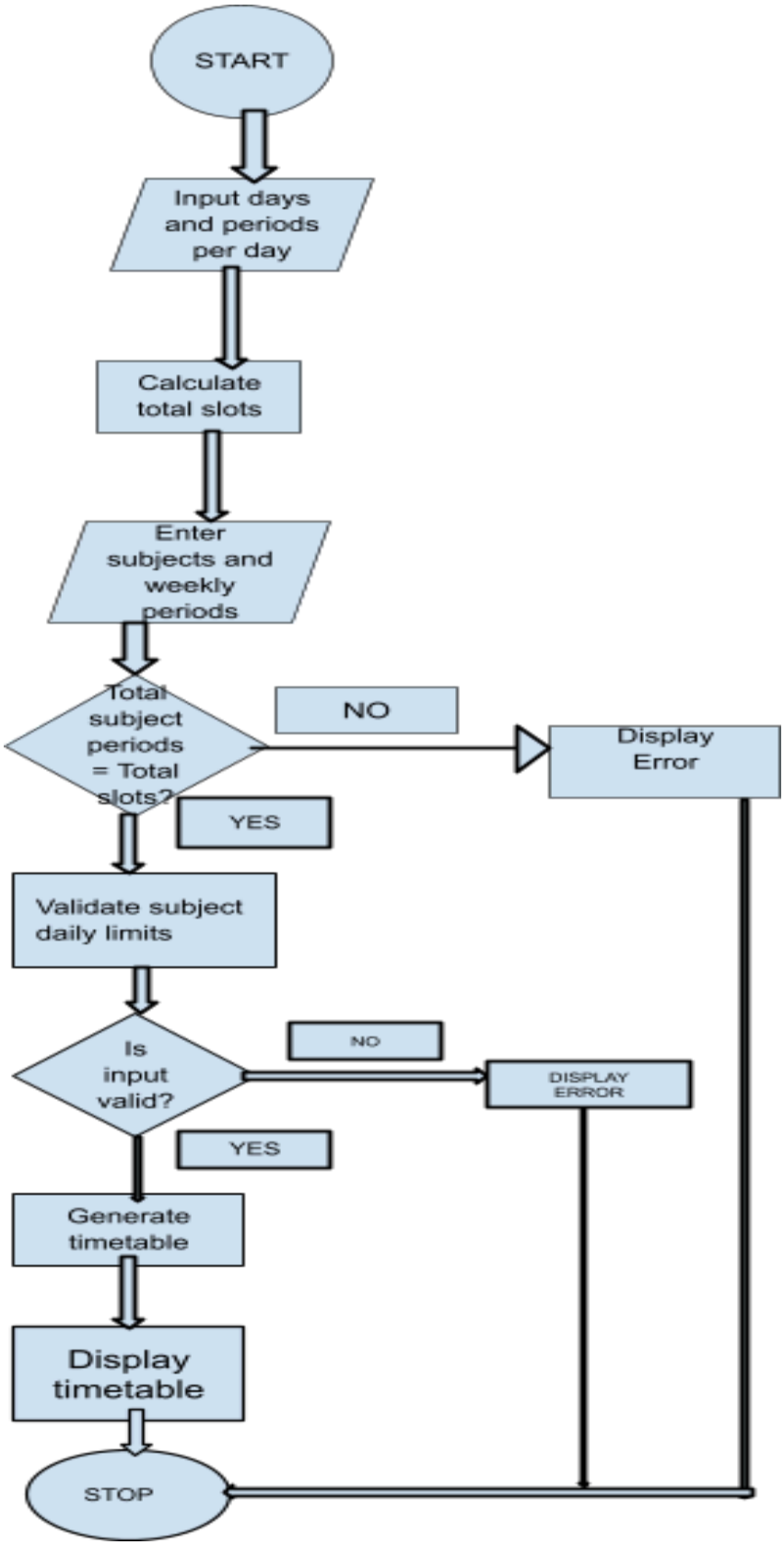
1. Operating System: Windows
2. Programming Language: Python 3
3. Code Editor: Visual Studio Code (VS Code)
4. Python Interpreter

5. ALGORITHM

The algorithm of the Classroom Timetable Generator is given below:

1. Start the program.
2. Enter the number of days.
3. Enter the number of periods per day.
4. Calculate the total number of timetable slots.
5. Enter the number of subjects.
6. Enter the subject names and their weekly periods.
7. Check whether the total subject periods are equal to the total timetable slots.
8. Check whether each subject can be scheduled within the given daily limit.
9. Select the subject having the highest number of remaining periods.
10. If two subjects have equal remaining periods, select the one entered first.
11. Assign the selected subject to a period, ensuring it is not scheduled more than twice in a day.
12. Repeat until all periods are filled.
13. Display the weekly timetable.
14. Stop the program.

6 . FLOWCHART


START
Input days and periods per day
Calculate total slots
Enter subjects and weekly periods
Total subject periods = Total slots?
NO
Display Error
YES
Validate subject daily limits
Is input valid?
NO
DISPLAY ERROR
YES
Generate timetable
Display timetable
STOP

7. SOURCE CODE

```
# Classroom Timetable Generator

days_name = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]

# Taking input
days = int(input("Enter number of days: "))
periods_per_day = int(input("Enter periods per day: "))

# Input validation
if days <= 0 or days > 6 or periods_per_day <= 0:
    print("Error: Invalid days or periods.")
    exit()

n = int(input("Enter number of subjects: "))

if n <= 0:
    print("Error: At least one subject is required.")
    exit()

# Dictionary to store subjects and periods
remaining = {}

for i in range(n):
    subject = input("Enter subject name: ")
    periods = int(input("Enter periods for " + subject + ": "))

    if subject in remaining or periods <= 0:
        print("Error: Invalid subject or periods.")
        exit()

    remaining[subject] = periods

# Check total periods
total_periods = sum(remaining.values())
total_slots = days * periods_per_day

if total_periods != total_slots:
    print("Error: Total weekly periods (" + str(total_periods) +
        ") must equal available slots (" + str(total_slots) + ").")
    exit()

# Check daily limit
for subject in remaining:
    if remaining[subject] > days * 2:
        print("Error: " + subject +
```

```
                    " needs more than 2 periods per day.")
                                exit()

                      # Generate timetable
                         timetable = []

                    for day in range(days):

                         daily_count = {}
                        daily_schedule = []

                for period in range(periods_per_day):

                 # Find subjects having remaining periods
                              available = [
                       subject for subject in remaining
                           if remaining[subject] > 0
                      and daily_count.get(subject, 0) < 2
                                    ]

                            if not available:
               print("Error: Timetable cannot be generated.")
                                exit()

            # Select subject with maximum remaining periods
          selected = max(available, key=lambda x: remaining[x])

                    daily_schedule.append(selected)

                        remaining[selected] -= 1

         daily_count[selected] = daily_count.get(selected, 0) + 1

                  timetable.append(daily_schedule)

                       # Print timetable
        print("\n========== WEEKLY TIMETABLE ==========")

                     for i in range(days):
         print(days_name[i] + ": " + ", ".join(timetable[i]))
```

8. TEST CASES AND OUTPUT

The program was tested using different input values to check whether it generates the correct timetable and handles invalid inputs properly.

Test Case 1: Normal Input

Days: 3

Periods per day: 3

Subjects:

- Maths: 3
- Physics: 2
- Programming: 3
- English: 1

Expected Result: A valid weekly timetable with 9 periods.

Test Case 2: Minimum Input

Days: 1

Periods per day: 2

Subject:

- Maths: 2

Expected Result: Both periods are assigned to Maths.

Test Case 3 (a) : Invalid Input

Days: 3

Periods per day: 3

Subject:

- Maths: 3
- Physics: 2
- Programming: 3

Expected Result: Error message because total periods do not match total slots.

Test Case 3 (b) : Invalid Input – Daily Period Limit

Days: 1

Periods per day: 3

Subjects:

- Maths: 3

Expected Result: Error message because a subject cannot be scheduled more than two times in a single day.

Test Case 4: Valid Input

Days: 2

Periods per day: 2

Subjects:

- Programming: 2
- English: 2

Expected Result: The timetable should be generated successfully, with each subject scheduled according to the given periods.

9. ACTUAL OUTPUT

Test case 1:

========== WEEKLY TIMETABLE ==========

Mon: maths, programming, maths

Tue: physics, programming, maths

Wed: physics, programming, english

Test case 2:

========== WEEKLY TIMETABLE ==========

Mon: maths, maths

Test case 3:

(a) Error: Total weekly periods (8) must equal available slots (9)

(b) Error: maths needs more than 2 periods per day.

Test case 4:

========== WEEKLY TIMETABLE ==========

Mon: programming, english

Tue: programming, english

10. CONCLUSION

The Classroom Timetable Generator project was successfully developed using Python 3.

This project automatically generates a weekly classroom timetable according to the given conditions. It uses Python concepts such as lists, dictionaries, loops, conditional statements, and scheduling logic.

The project helps reduce manual work and saves time. It also checks input conditions and displays appropriate error messages when invalid data is entered.

Through this project, I learned how to apply Python programming concepts to solve a real-world problem.

11. FUTURE SCOPE

The Classroom Timetable Generator can be improved in the future by adding more features.

1. A graphical user interface (GUI) can be developed.
2. The program can support more subjects and classes.
3. Teacher availability can be considered while generating timetables.
4. Classroom availability can be managed automatically.
5. The timetable can be saved as a PDF or Excel file.
6. A database can be added to store timetable information.